**c:\hpc-23-feb-2026\.\18\eg148.c**

```
#include<pthread.h>
#include<stdio.h>
#include<unistd.h>
void * do_something(void *ptr)
{
int i;
for(i=1;i<=10;++i)
{
printf("%d\n",i);
sleep(1);
}
}
int main()
{
pthread_t tid;
pthread_create(&tid,NULL,do_something,NULL);
pthread_detach(tid);
sleep(5);
printf("main woke up and is about to end\n");
return 0;
}
```

Thinking Machines Classroom Codes

**c:\hpc-23-feb-2026\.\18\eg149.c**

```
#include<pthread.h>
#include<stdio.h>
#include<unistd.h>
void * do_something(void *ptr)
{
int i;
for(i=1;i<=10;++i)
{
printf("%d\n",i);
sleep(1);
}
}
int main()
{
pthread_t tid;
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr,PTHREAD_CREATE_DETACHED); // JOINABLE
pthread_create(&tid,&attr,do_something,NULL);
sleep(5);
printf("main woke up and is about to end\n");
pthread_attr_destroy(&attr);
return 0;
}
```

**c:\hpc-23-feb-2026\.\18\eg150.c**

```
#include<pthread.h>
#include<stdio.h>
void * do_something(void *ptr)
{
int *x;
int e;
x=(int *)ptr;
for(e=1;e<=1000;++e)
{
(*x)++;
}
return NULL;
}
int main()
{
int num;
pthread_t tid1,tid2;
num=0;
pthread_create(&tid1,NULL,do_something,(void *)&num);
pthread_create(&tid2,NULL,do_something,(void *)&num);
pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
printf("%d\n",num);
}
```

Thinking Machines Classroom Codes

**c:\hpc-23-feb-2026\.\18\eg151.c**

```
#include<pthread.h>
#include<stdio.h>
void * do_something(void *ptr)
{
int *x;
int e;
x=(int *)ptr;
for(e=1;e<=100000;++e)
{
(*x)++;
}
return NULL;
}
int main()
{
int num;
pthread_t tid1,tid2;
num=0;
pthread_create(&tid1,NULL,do_something,(void *)&num);
pthread_create(&tid2,NULL,do_something,(void *)&num);
pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
printf("%d\n",num);
}
```

Thinking Machines Classroom Codes

**c:\hpc-23-feb-2026\.\18\eg152.c**

```
// just by declaring _Atomic, the issue is not resolved,
// we need to declare the pointer with _Atomic
#include<pthread.h>
#include<stdio.h>
#include<stdatomic.h>
void * do_something(void *ptr)
{
_Atomic int *x;
int e;
x=(_Atomic int *)ptr;
for(e=1;e<=100000;++e)
{
(*x)++;
}
return NULL;
}
int main()
{
_Atomic int num;
pthread_t tid1,tid2;
num=0;
pthread_create(&tid1,NULL,do_something,(void *)&num);
pthread_create(&tid2,NULL,do_something,(void *)&num);
pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
printf("%d\n",num);
}
```

**c:\hpc-23-feb-2026\.\18\eg153.c**

```
// just by declaring _Atomic, the issue is not resolved,
// we need to declare the pointer with _Atomic
#include<pthread.h>
#include<stdio.h>
#include<stdatomic.h>
void * do_something(void *ptr)
{
_Atomic int *x;
int e;
x=(_Atomic int *)ptr;
for(e=1;e<=100000;++e)
{
(*x)+=1;
}
return NULL;
}
int main()
{
_Atomic int num;
pthread_t tid1,tid2;
num=0;
pthread_create(&tid1,NULL,do_something,(void *)&num);
pthread_create(&tid2,NULL,do_something,(void *)&num);
pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
printf("%d\n",num);
}
```

**c:\hpc-23-feb-2026\.\18\eg154.c**

```
// this is incorrect, will not work as desirned
#include<pthread.h>
#include<stdio.h>
#include<stdatomic.h>
void * do_something(void *ptr)
{
_Atomic int *x;
int e;
x=(_Atomic int *)ptr;
for(e=1;e<=100000;++e)
{
(*x)=(*x)+1;  // not an atomic operation as it is in case of ++ or +=1
}
return NULL;
}
int main()
{
_Atomic int num;
pthread_t tid1,tid2;
num=0;
pthread_create(&tid1,NULL,do_something,(void *)&num);
pthread_create(&tid2,NULL,do_something,(void *)&num);
pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
printf("%d\n",num);
}
```

**c:\hpc-23-feb-2026\.\18\eg155.c**

```
#include<pthread.h>
#include<stdio.h>
#include<stdatomic.h>
void * do_something(void *ptr)
{
_Atomic int *x;
int e;
x=(_Atomic int *)ptr;
for(e=1;e<=100000;++e)
{
(*x)+=1;
}
return NULL;
}
int main()
{
int num;
pthread_t tid1,tid2;
num=0;
pthread_create(&tid1,NULL,do_something,(void *)&num);
pthread_create(&tid2,NULL,do_something,(void *)&num);
pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
printf("%d\n",num);
}
```

**c:\hpc-23-feb-2026\.\18\eg156.cpp**

```
#include<iostream>
using namespace std;
// lambda expressions (technically an anonymous function, whose address is assigned
//  to a pointer)
int main()
{
auto a=[](){
cout<<"great"<<endl;
};
auto b=[](int p,int q){
cout<<p+q<<endl;
};
auto c=[](int p,int q)->int{
return p+q;
};
int j=10;
auto d=[j](){
cout<<j<<endl;
//j=50; // will not compile, j is read only type as per capture statement
};
auto e=[&j](){
cout<<j<<endl;
j=50;
};
a();
b(10,20);
cout<<c(100,200)<<endl;
d();
e();
cout<<j<<endl;
return 0;
}
```

**c:\hpc-23-feb-2026\.\18\eg157.cpp**

```
#include<thread>
#include<iostream>
using namespace std;
int main()
{
int num=0;
auto a=[&num](){
for(int i=1;i<=100000;++i)
{
num+=1;
}
};
thread t1(a);
thread t2(a);
t1.join();
t2.join();
cout<<num<<endl;
return 0;
}
```

Thinking Machines Classroom Codes

**c:\hpc-23-feb-2026\.\18\eg158.cpp**

```
#include<thread>
#include<iostream>
#include<atomic>
using namespace std;
int main()
{
atomic<int> num(0);
auto a=[&num](){
for(int i=1;i<=100000;++i)
{
num+=1;
}
};
thread t1(a);
thread t2(a);
t1.join();
t2.join();
cout<<num<<endl;
return 0;
}
```

Thinking Machines Classroom Codes